

Guida — installare sul Mac e usare su una wifi senza internet

Pico MultiView **non ha bisogno di internet per funzionare**: parla con i visori solo sulla rete locale. Internet serve **una volta sola**, per scaricare e installare il programma. Quindi si prepara tutto dove internet c'è (casa, ufficio) e all'evento basta una wifi qualsiasi, anche isolata dal mondo.

1. Installare da zero

2. Preparare i visori

2b. Mettere i filmati

3. La wifi senza internet

4. Il Mac sulla wifi

5. All'evento

6. Prova generale

7. Se qualcosa non va

1. Installare su un Mac nuovo — da zero, con internet

Vale per un Mac su cui non è mai stato installato niente. Si fa **una volta sola, con internet**: alla fine in *Applicazioni* c'è un'app che contiene già tutto e non scarica più niente. Ci vogliono circa 20 minuti, quasi tutti di attesa.

Ogni riquadro grigio va copiato e incollato nel **Terminale**, poi **Invio**. Il Terminale si apre con **⌘+Spazio**, scrivendo «Terminale».

Il modo più rapido: un comando solo

Incolla nel Terminale **queste due righe insieme** e premi **Invio**. Fanno da sole tutti i passi qui sotto: installano Node.js, scaricano il progetto in *Documenti*, scaricano i componenti e mettono l'app in *Applicazioni*. Non servono **git** né un account GitHub.

```
U=https://raw.githubusercontent.com/fosforoscienza/pico-multiview/main
/bin/bash -c "$(curl -fsSL $U/installa-mac.sh)"
```

Ci vogliono 5-15 minuti. A metà chiede la **password del Mac** (solo se deve installare Node.js; mentre la scrivi non si vede niente). Alla fine scrive *Fatto.* in verde e apre la cartella con l'app. Si può rilanciare quando si vuole: aggiorna il progetto e ricostruisce l'app.

Se il comando unico arriva in fondo, salta pure al punto 2. Se si ferma, il messaggio rosso dice dove: i passi qui sotto fanno la stessa cosa a mano, uno per volta.

Passo 1 — Gli strumenti di Apple (contengono **git**)

```
xcode-select --install
```

Compare una finestra: **Installa** → **Accetta** e aspetta la fine (5-10 minuti). Se dice che sono già installati, va bene così. Per controllare:

```
git --version
```

Passo 2 — Node.js

Scarica e installa l'ultima versione LTS dal sito ufficiale (va bene sia per i Mac Apple sia per gli Intel):

```
BASE=https://nodejs.org/dist/latest-v24.x
NOME=$(curl -fsSL $BASE/SHASUMS256.txt | awk '/\.pkg$/ {print $2}')
echo "Scarico $NOME"
curl -fL -o ~/Downloads/node.pkg "$BASE/$NOME"
sudo installer -pkg ~/Downloads/node.pkg -target /
```

La terza riga deve scrivere qualcosa come *Scarico node-v24.21.0.pkg*: se dopo «Scarico» non c'è niente, il Mac non ha raggiunto `nodejs.org` (controlla internet e rilancia le righe). L'ultima chiede la **password del Mac**: mentre la scrivi non si vede niente, è normale. Alla fine deve dire *The install was successful*. Poi controlla:

```
node -v
npm -v
```

Copia le righe dalla Guida dell'app o dal README: dal PDF, se una riga va a capo, il comando arriva spezzato. Se resta un errore, scarica il pacchetto a mano: da `nodejs.org`, pulsante **LTS** → *macOS Installer (.pkg)*, doppio clic e avanti fino alla fine. Su macOS più vecchio di 13.5 Node 24 non si installa: nella prima riga scrivi `latest-v22.x` al posto di `latest-v24.x`.

Passo 3 — Scaricare il progetto

```
cd ~/Documents
git clone https://github.com/fosforoscienza/pico-multiview.git
```

Il progetto è pubblico: non serve nessun account GitHub, né nome utente né password. Crea la cartella `pico-multiview` dentro *Documenti*. Più avanti, per aggiornarlo all'ultima versione, basta `git pull` da dentro quella cartella.

In alternativa, senza `git` (e quindi senza il passo 1): scarica il progetto come archivio ZIP.

```
cd ~/Downloads
curl -fL -o pico-multiview.zip \
  https://github.com/fosforoscienza/pico-multiview/archive/refs/heads/main.zip
unzip -q pico-multiview.zip
mv pico-multiview-main ~/Documents/pico-multiview
```

Lo stesso ZIP si scarica anche dal browser: sulla pagina del progetto *Code* → *Download ZIP*. Safari lo estrae da solo in *Download*; in quel caso servono solo le ultime due righe qui sotto:

```
mv ~/Downloads/pico-multiview-main ~/Documents/pico-multiview
xattr -dr com.apple.quarantine ~/Documents/pico-multiview
```

Passo 4 — I componenti del programma

```
cd ~/Documents/pico-multiview
npm install
npm run deps:adb
```

Il primo scarica Electron (circa 220 MB) e il componente per il video, il secondo `adb`, lo strumento che parla con i visori. Scorre molto testo: è normale. Gli avvisi gialli `npm warn` si possono ignorare, conta che alla fine non ci sia `npm error`.

Passo 5 — Creare l'app e metterla in Applicazioni

```
cd ~/Documents/pico-multiview
npm run dist
open dist/*-$(uname -m | sed 's/x86_64/x64/').dmg
```

Ci vogliono un paio di minuti. Si apre una finestra: trascina **Pico MultiView** nella cartella *Applicazioni*. Costruita su questo stesso Mac si apre con un doppio clic, senza avvisi di sicurezza.

Passo 6 — Controllare che funzioni

Apri **Pico MultiView** da *Applicazioni* (o dal Launchpad). Per vederla al lavoro anche senza visori, con dieci visori finti:

```
cd ~/Documents/pico-multiview
npm start -- --demo=10
```

Fatto: l'app in *Applicazioni* contiene già `adb` e il componente video, non scarica niente all'avvio e funziona anche senza internet. I file `.dmg` in `dist/` si possono copiare su altri Mac (chiavetta, AirDrop): lì basta aprirli e trascinare l'app, più la riga di sblocco qui sotto.

I singoli software, uno per uno — se un download automatico non va

I passi sopra scaricano tutto da soli. Se uno si blocca (rete lenta, un sito che non risponde), ogni pezzo si può scaricare e mettere al suo posto anche a mano.

Strumenti di Apple (`git`)

Se `xcode-select --install` dice «non disponibile sul server»: dal sito developer.apple.com/download/all (serve un Apple ID, anche gratuito) scarica *Command Line Tools for Xcode*, apri il `.dmg` e fai doppio clic sul `.pkg`. Non servono se il progetto lo scarichi come ZIP.

Node.js

Dal sito nodejs.org, pulsante **LTS** → *macOS Installer (.pkg)*, poi doppio clic sul file e avanti fino alla fine. È lo stesso pacchetto del passo 2. In alternativa, con Homebrew: `brew install node`.

`adb` (Android platform-tools, di Google)

Va nella cartella `vendor/platform-tools` del progetto:

```
cd ~/Downloads
curl -fLO https://dl.google.com/android/repository/platform-tools-latest-darwin.zip
mkdir -p ~/Documents/pico-multiview/vendor
unzip -o -q platform-tools-latest-darwin.zip -d ~/Documents/pico-multiview/vendor
xattr -dr com.apple.quarantine ~/Documents/pico-multiview/vendor/platform-tools
~/Documents/pico-multiview/vendor/platform-tools/adb version
```

L'ultima riga deve rispondere *Android Debug Bridge version* e un numero. In alternativa, con Homebrew: `brew install --cask android-platform-tools`: l'app lo trova da sola, ma solo su questo Mac — il `.dmg` da passare ad altri contiene `adb` soltanto se sta in `vendor/platform-tools`.

`scrcpy-server` 2.7 (lo streaming video dai visori)

Va nel file `vendor/scrcpy-server`, senza estensione. Serve **proprio la versione 2.7**:

```
mkdir -p ~/Documents/pico-multiview/vendor
curl -fL -o ~/Documents/pico-multiview/vendor/scrcpy-server \
  https://github.com/Genymobile/scrcpy/releases/download/v2.7/scrcpy-server-v2.7
shasum -a 256 ~/Documents/pico-multiview/vendor/scrcpy-server
```

Il codice che compare deve iniziare con `a23c5659f36c` e finire con `01377a1adba`. Se è diverso il file è rovinato: cancellalo e ricaricalo.

Electron (la finestra del programma)

Lo scarica `npm install` (circa 220 MB) e non c'è un modo comodo di farlo a mano. Se si interrompe, rilancialo: riprende da dove serve. Se insiste a fallire, cancella la cartella e riprova da capo:

```
cd ~/Documents/pico-multiview
rm -rf node_modules
npm install
```

Messi a mano `adb` e `scrcpy-server`, si riprende dal passo 4 (`npm install` li trova già al loro posto e non li scarica) e poi il passo 5.

Su un altro Mac, con il `.dmg` già pronto

Il `.dmg` si può passare con chiavetta o AirDrop, oppure scaricare dalla pagina *Releases* del progetto (`github.com/fosforoscienza/pico-multiview/releases`), se lì c'è quello della versione che ti serve.

`arm64` per i Mac con chip Apple (M1, M2, M3, M4...), `x64` per gli Intel (menu → *Informazioni su questo Mac*). Trascina l'app in *Applicazioni*, poi una volta sola, perché arrivata da fuori non è firmata da Apple:

```
xattr -dr com.apple.quarantine "/Applications/Pico MultiView.app"
```

All'evento non lanciare `git pull`, `npm install` o `npm run dist`: servono internet. Gli aggiornamenti si fanno prima di partire, ripetendo i passi 4 e 5 dopo un `git pull`.

2. Preparare i visori — una volta sola per visore

Non serve internet: basta il cavo USB-C. Conviene farlo con calma prima dell'evento.

1. Sul visore: *Impostazioni* → *Generale* → *Informazioni sul dispositivo* → tocca **Versione software** 7-8 volte. Poi *Impostazioni* → *Sviluppatore* → attiva **Debug USB**.
2. Collega il visore al Mac con un cavo USB-C **dati**. Nel visore metti la spunta su «**Consenti sempre da questo computer**» e conferma.

3. Collega il visore alla wifi dell'evento (vedi punto 3), poi nell'app clicca una postazione vuota → **Adotta quello collegato via USB**. Il cavo si può staccare.
4. Dai un nome alla postazione (⚙ sulla miniatura) e attacca un'etichetta uguale sul visore.

L'autorizzazione vale per *questo* Mac. La procedura completa è in [docs/SETUP-PICO.md](#).

2b. Mettere i filmati sui visori

Il programma manda in riproduzione i filmati che **stanno già dentro i visori**: non li trasmette dal Mac. Vanno copiati prima, su ogni visore, in una cartella qualsiasi della memoria — per esempio **Movies** o **Download**, ma **non dentro Android**, che la ricerca salta. Formati riconosciuti: mp4, mkv, webm, mov, m4v, avi, 3gp, inv.

Con i visori già collegati al programma (in wifi o col cavo), dal Terminale. Incolla tutte le righe insieme: le prime tre trovano **adb**, nella cartella del progetto o dentro l'app; le ultime copiano il filmato su tutti i visori collegati, uno dopo l'altro.

```
ADB=~/Documents/pico-multiview/vendor/platform-tools/adb
APP="/Applications/Pico MultiView.app/Contents/Resources"
[ -x "$ADB" ] || ADB="$APP/app.asar.unpacked/vendor/platform-tools/adb"
for s in $("ADB" devices | awk 'NR>1 && $2=="device" {print $1}'); do
  "$ADB" -s "$s" push ~/Movies/filmato.mp4 /sdcard/Movies/
done
```

Al posto di **~/Movies/filmato.mp4** metti il tuo file: puoi trascinarlo dal Finder dentro la finestra del Terminale, e il percorso si scrive da solo. Per ogni visore compare una riga con *file pushed*. Sulla wifi un filmato lungo ci mette qualche minuto per visore; col cavo USB è molto più veloce.

Poi nel programma: **Video...** → **Rileggi i file**. Se un visore non ha il filmato, la finestra lo dice sotto il titolo, visore per visore.

3. La wifi senza internet

- Basta un **router wifi** (anche un router da viaggio) acceso e **non collegato a internet**. Il suo compito è solo dare un indirizzo a ogni apparecchio e metterli in contatto fra loro.
- Nel router **disattiva l'isolamento client** (*AP isolation, client isolation*) e non usare la rete ospiti: altrimenti il Mac non vede i visori.
- Stesso nome di rete e stessa password per tutti i visori. Meglio la banda a 5 GHz.
- Consigliato: nel router **riserva un indirizzo fisso** a ogni visore (*prenotazione DHCP*). Così ogni visore ha sempre lo stesso IP e le postazioni si ricollegano subito.
- Sui visori: collegali a quella wifi e **dimentica le altre reti salvate**, così non ci saltano sopra. Se il visore avvisa che la rete non ha internet, scegli di restare collegato.
- Per non dover rifare il giro col cavo dopo ogni riavvio di un visore, rendi permanente la connessione wifi di **adb** (col visore collegato via USB):

```
adb -s <seriale> shell setprop persist.adb.tcp.port 5555
```

4. Il Mac sulla wifi senza internet

- Collega il Mac alla wifi dell'evento. macOS scrive «*Nessuna connessione a Internet*»: è normale, ignoralo.
- Evita che il Mac salti da solo su un'altra rete conosciuta (o sull'hotspot del telefono): *Impostazioni di Sistema* → *Wi-Fi* → *Dettagli...* accanto alle altre reti → togli **Accedi automaticamente**.
- Se ti serve internet sul Mac nello stesso momento, prendilo da un cavo Ethernet: la wifi resta ai visori.
- La prima volta macOS chiede se Pico MultiView può *trovare dispositivi sulla rete locale*: rispondi **Consenti**. Senza quel permesso il programma non raggiunge i visori in wifi — vede solo quelli col cavo. Se hai risposto di no, o la domanda non è comparsa: *Impostazioni di Sistema* → *Privacy e sicurezza* → *Rete locale*, attiva **Pico MultiView**, poi chiudi e riapri il programma.
- Se macOS chiede se consentire a Pico MultiView di *accettare connessioni in entrata*, rispondi **Consenti**: serve al telecomando dall'iPad.

5. All'evento

1. Accendi il router e aspetta che la wifi compaia.
2. Accendi i visori, poi collega il Mac alla stessa wifi.
3. Apri Pico MultiView (da *Applicazioni*, oppure con l'icona nella cartella del progetto). Le postazioni si ricollegano da sole; se ne manca qualcuna premi **Cerca in rete**.
4. Telecomando dall'iPad: iPad sulla stessa wifi, sul Mac **Telecomando...** → **Accendi**, e inquadra il QR con la fotocamera dell'iPad: si apre già sbloccato. In alternativa scrivi in Safari l'indirizzo che compare (per esempio `http://192.168.1.10:8788/?k=PIN`). Anche questo funziona senza internet.

6. Prova generale — prima di partire

Il modo sicuro di sapere che all'evento andrà: **stacca il cavo internet del router** (la presa WAN) e prova tutto così.

- ☐ l'app si apre e in basso non compaiono errori rossi nel **Log**;
- ☐ tutti i visori compaiono nelle loro postazioni, con l'immagine;
- ☐ un clic su una miniatura apre l'anteprima grande;
- ☐ **Video...** fa partire un filmato su tutti, e la barra del filmato lo mette in pausa e lo ferma;
- ☐ l'iPad si collega al telecomando;
- ☐ dopo aver spento e riacceso un visore, torna da solo.

Se funziona a casa senza internet, funziona anche all'evento.

7. Se qualcosa non va

«Cerca in rete» non trova nessun visore

Se l'avviso parla del permesso «Rete locale», è quello: vedi sopra. Altrimenti: visori accesi e svegli, sulla stessa wifi del Mac (controlla che il Mac non sia saltato su un'altra rete), isolamento client spento nel router. Se la rete del router non è una /24 standard (come `192.168.1.x`), indica le sottoreti in `config.json` → `scan.subnets`. Puoi anche inserire l'IP a mano dalla postazione.

«Video...» non trova nessun filmato

Sotto il titolo la finestra dice, visore per visore, quanti file ha trovato e dove ha cercato, oppure perché non ha potuto cercare. «Nessun file video»: i filmati non sono sul visore, sono dentro la cartella `Android` o hanno un formato non riconosciuto — copiali come al punto 2b. «Non collegato»: il visore non risponde, premi **Cerca in rete**. «Nessun visore nelle postazioni»: assegna i visori alle postazioni, la ricerca guarda solo quelli.

«Adotta USB» non riesce, e staccando il cavo il visore sparisce

Il visore non è passato al wifi: la finestra dell'esito dice perché, e finché non ci riesce il cavo va lasciato. I casi: **reti diverse** — il Mac è su un'altra wifi, collegalo a quella del visore; **Rete locale** — macOS non lascia il programma parlare con la rete: *Impostazioni di Sistema* → *Privacy e sicurezza* → *Rete locale*, attiva Pico MultiView e riapri il programma; **nessuna risposta** — il router isola i dispositivi fra loro (rete ospiti, «isolamento client»): disattivalo o usa un'altra rete. Sistemato, ripremi **Adotta USB**.

Un visore risulta `unauthorized`

Non era stata messa la spunta «Consenti sempre», oppure è un altro Mac. Ricollegalo col cavo e conferma nel visore.

Dopo un riavvio un visore non si collega più

Ricollegalo via USB e premi **Adotta USB**; per non doverlo rifare usa il comando `persist.adb.tcp.port` del punto 3.

«adb non eseguibile» o «scrcpy-server mancante»

Al passo 4 non è stato completato `npm run deps:adb` mentre c'era internet (oppure metti i due file a mano, come al punto 1). Serve una connessione per un minuto: anche l'hotspot del telefono va bene, poi torni alla wifi dell'evento.

Altro

Apri il **Log** in basso a sinistra: dice visore per visore cosa è successo. Con **Copia** lo mandi a chi ti aiuta.

Per provare l'app senza visori: dai sorgenti, `npm start -- --demo=10`.